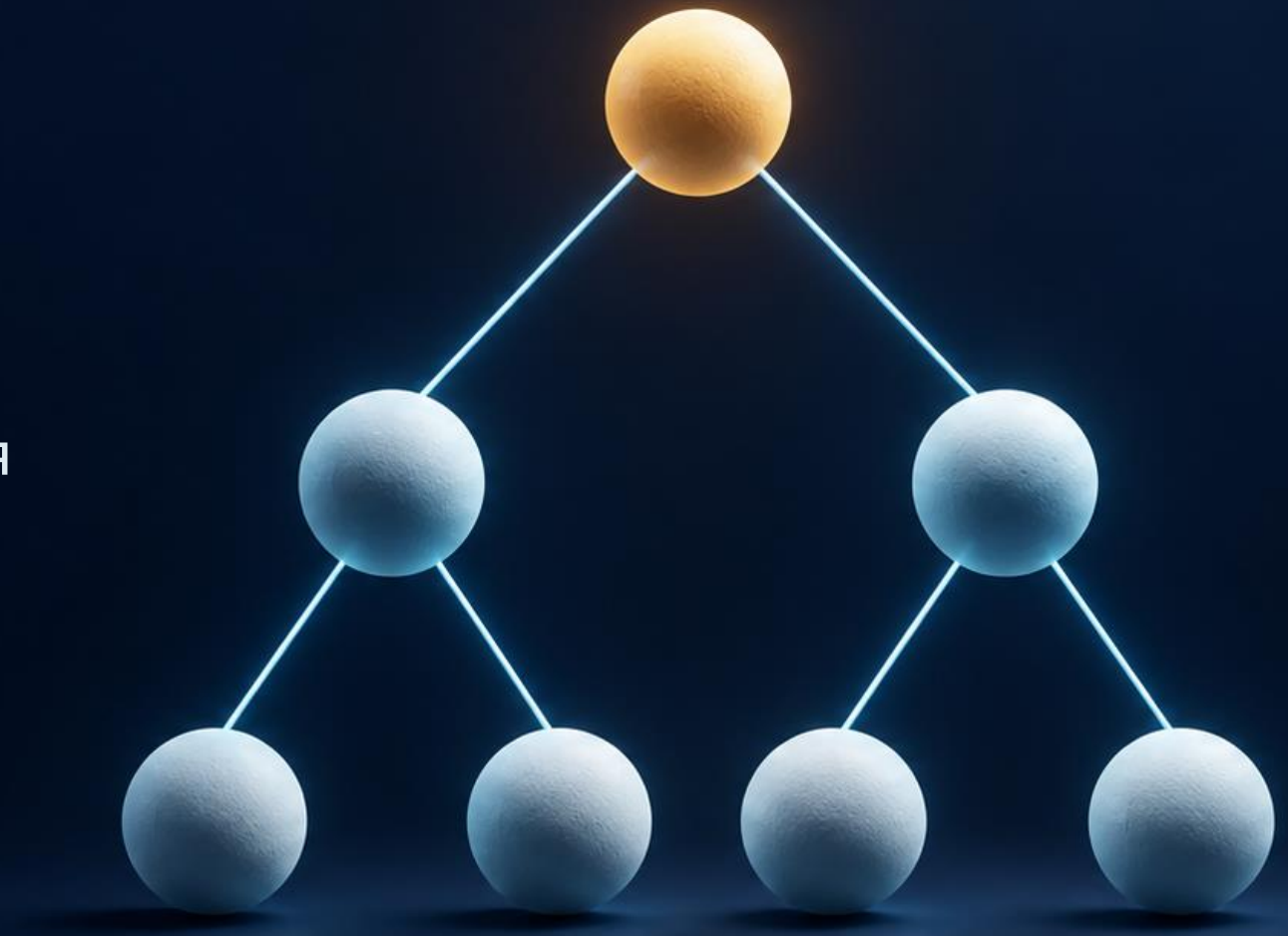
Куча

Бинарная куча, способы хранения
и heapq в Python



Структуры данных

Очередь с приоритетом

Каждый элемент имеет приоритет. Извлечение возвращает элемент с минимальным или максимальным приоритетом.

Основные операции

push(x) — добавить элемент

peek() — посмотреть вершину

extractMin() — извлечь минимум

Пример

(1, «срочно»)

(3, «обычно»)

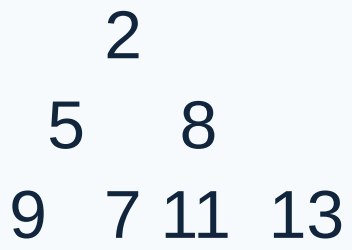
(5, «позже»)

Следующим выйдет элемент с приоритетом 1.

Бинарная куча

Структура

Полное бинарное дерево: все уровни, кроме последнего, заполнены; последний заполняется слева направо.



Порядок min-heap

Значение родителя ≤ значений его детей. Поэтому в корне находится минимум.

Узлы 5 и 8 между собой не обязаны быть отсортированы.

Для max-heap знак меняется на ≥, а в корне находится максимум.

Добавление и извлечение

push(x): просеивание вверх

1. Поставить x в конец последнего уровня.
2. Пока x меньше родителя, обменивать их.
3. Остановиться у корня или при верном порядке.

extractMin(): просеивание вниз

1. Забрать корень.
2. Перенести на его место последний элемент.
3. Менять его с меньшим ребёнком, пока нужно.

Пример: push(1)

[2, 5, 8, 9, 7] → [1, 5, 2, 9, 7, 8]

Обе операции проходят не более высоты дерева: O(log n).

Сложность операций

Операция	Время
Посмотреть минимум	**O(1)**
Добавить элемент	**O(log n)**
Извлечь минимум	**O(log n)**
Построить кучу из n элементов	**O(n)**

O(n) для построения получается при просеивании вниз от последних внутренних узлов.

Хранение в массиве

Узлы записываются по уровням слева направо. Отдельные указатели на детей не нужны.

Индексы при нумерации с нуля

parent(i) = (i − 1) // 2
left(i) = 2i + 1
right(i) = 2i + 2

Пример

i: 0 1 2 3 4 5 6
a: 2 5 8 9 7 11 13

Массив даёт O(1) доступ к родителю и детям по индексу.

Связная реализация и список

Бинарную кучу можно хранить узлами с полями value, left, right, parent. При добавлении всё равно нужно найти следующую свободную позицию полного дерева.

Если использовать обычный связный список как очередь с приоритетом

Структура	push	extractMin
Неотсортированный список	O(1)	O(n)
Отсортированный список	O(n)	O(1)
Бинарная куча в массиве	**O(log n)**	**O(log n)**

Python: list и куча

Python list — динамический массив. Модуль heapq поддерживает свойство кучи прямо внутри обычного списка.

Ключевые отличия

Корень находится в h[0].

Элементы списка не обязаны идти по возрастанию.

Для изменений используйте функции heapq.

```
h = [2, 5, 8, 9, 7, 11, 13]  # корректная min-heap
```

Python: модуль heapq

```
import heapq

h = [7, 2, 9, 4]
heapq.heapify(h)
heapq.heappush(h, 1)
x = heapq.heappop(h)  # 1
y = h[0]              # 2
```

Стоимость

heapify O(n)

heappush O(log n)

heappop O(log n)

h[0] O(1)

В Python 3.14 появились heapify_max, heappush_max и heappop_max.

Где помогает куча

k-й наибольший элемент

Держим min-heap размера k. После просмотра всех n элементов её корень равен k-му наибольшему.

Время: O(n log k)
Память: O(k)

Слияние k отсортированных последовательностей

Храним в куче первый ещё не взятый элемент каждой последовательности. После извлечения добавляем следующий из той же последовательности.

Для N элементов: O(N log k)
В Python: heapq.merge(...)

Куча полезна, когда нужен лучший текущий элемент, а полная сортировка не требуется.